

HDOC Day-28 - Solutions

1) Variable-length 2D array: input column-by-column and print in row-major order

This program repeatedly asks for positive row and column values, creates a variable-length 2D array, reads the elements column by column, and finally prints all elements in row-major order using one loop and a linear index.

```
#include <stdio.h>

int main()
{
    int rows, cols;

    /* Read positive dimensions */
    do {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
    } while (rows <= 0);

    do {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
    } while (cols <= 0);

    /* Variable-length 2D array */
    int a[rows][cols];

    /* Input column by column */
    printf("Enter %d elements:\n", rows * cols);
    for (int j = 0; j < cols; j++) {
        for (int i = 0; i < rows; i++) {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Print in row-major order using a single loop */
    printf("\nElements in row-major order:\n");
    for (int k = 0; k < rows * cols; k++) {
        int i = k / cols;
        int j = k % cols;
        printf("%d ", a[i][j]);
    }

    printf("\n");
    return 0;
}
```

Key idea: For a linear index **k**, the row is **k / cols** and the column is **k % cols**.

2) Copy a 2D VLA into a 1D array in row-major order

This program reads a variable-length 2D array, copies its elements into a 1D array in row-major order, and displays both arrays.

```
#include <stdio.h>

int main()
{
    int rows, cols;

    do {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
    } while (rows <= 0);

    do {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
    } while (cols <= 0);

    int a[rows][cols];
    int b[rows * cols];

    /* Read 2D array */
    printf("Enter %d elements:\n", rows * cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Copy to 1D array in row-major order */
    for (int k = 0; k < rows * cols; k++) {
        int i = k / cols;
        int j = k % cols;
        b[k] = a[i][j];
    }

    /* Display 2D array */
    printf("\n2D Array:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    /* Display 1D array */
    printf("\n1D Array:\n");
    for (int k = 0; k < rows * cols; k++)
        printf("%d ", b[k]);

    printf("\n");
    return 0;
}
```

Row-major order means all elements of the first row are stored first, followed by the second row, and so on.